周报6_二叉树

题目

洛谷：普及×1;

力扣：简单×4 中等×11 困难×1;

- 基础数学
    1. B4061 位运算 2 (*)
- 二叉树
    1. 102. 二叉树的层序遍历(**)
    2. 103. 二叉树的锯齿形层序遍历(**)
    3. 662. 二叉树最大宽度(**)
    4. 104. 二叉树的最大深度(*)
    5. 111. 二叉树的最小深度(*)
    6. 297. 二叉树的序列化与反序列化(***)
    7. 105. 从前序与中序遍历序列构造二叉树(**)
    8. 958. 二叉树的完全性检验(**)
    9. 222. 完全二叉树的节点个数(*)
    10. 236. 二叉树的最近公共祖先(**)
    11. 235. 二叉搜索树的最近公共祖先(**)
    12. 113. 路径总和 II(**)
    13. 110. 平衡二叉树(*)
    14. 98. 验证二叉搜索树(**)
    15. 669. 修剪二叉搜索树(**)
    16. 337. 打家劫舍 III(**)

代码

1）基础数学

1. B4061 位运算 2 (*)

```cpp
#include <iostream>
#include <climits>

using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long a, b, ans;

    cin >> a >> b;
```

```
    cout << (a << b) << '\n';
    cout << (a >> b) << '\n';
    cout << ((a >> b) & 1) << '\n';
    // 将 a 的二进制的第 b 位设置为 0 后得到的十进制结果
    ans = a & ~(1LL << b);
    cout << ans << '\n';
    // 将 a 的二进制的第 b 位设置为 1 后得到的十进制结果;
    cout << (a | (1LL << b)) << '\n';
    // 将 a 的二进制的第 b 位取反后得到的十进制结果
    ans = a ^ (1LL << b);
    cout << ans << '\n';
    return 0;
}
```

2）二叉树

1. 102. 二叉树的层序遍历(**)

> 广度优先搜索(BFS)

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    enum {
        MAXN = 2001
    };

    vector<TreeNode*> que;
    int l, r;

    vector<vector<int>> levelOrder(TreeNode* root) {
        vector<vector<int>> ans;
        if (root != nullptr) {
            l = r = 0;
            que.resize(MAXN);
            que[r++] = root;
            // 队列内还有树节点
            while (l < r) {
                // 把上一层树的节点全部弹出
                int size = r - l;
```

```
                vector<int> inans;
                for (int i = 0; i < size; i++) {
                    // 队列头弹出
                    TreeNode* cur = que[l++];
                    inans.push_back(cur->val);
                    // 若当前节点存在左右子树，则进入队列
                    if (cur->left != nullptr) {
                        que[r++] = cur->left;
                    }
                    if (cur->right != nullptr) {
                        que[r++] = cur->right;
                    }
                }
                ans.push_back(inans);
            }
        }
        return ans;
    }
};
```

2. 103. 二叉树的锯齿形层序遍历(**)

> 蛇形层序遍历二叉树

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
right(right) {}
 * };
 */
class Solution {
public:
    enum {MAXN = 2001};

    int l, r;
    vector<TreeNode*> que;

    vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
        vector<vector<int>> ans;
        if (root != nullptr) {
            que.resize(MAXN);
            l = r = 0;
            bool reverse = false;
            que[r++] = root;
            int size;
            while (l < r) {
                size = r - l;
```

```
                vector<int> inans;
                // 把当前层节点值按序加入一维数组
                for (int i = reverse ? r - 1 : l, j = reverse ? -1: 1, k = 0;
                k < size; i += j, k++) {
                    inans.push_back(que[i]->val);
                }
                // 队列中弹出当层节点
                for (int i = 0; i < size; i++) {
                    TreeNode* cur = que[l++];
                    if (cur->left != nullptr) {
                        que[r++] = cur->left;
                    }
                    if (cur->right != nullptr) {
                        que[r++] = cur->right;
                    }
                }
                reverse = !reverse;
                ans.push_back(inans);
            }
        }
        return ans;
    }
};
```

3. 662. 二叉树最大宽度(**)

> 每一层的 宽度 被定义为该层最左和最右的非空节点（即，两个端点）之间的长度。将这个二叉树视作与满二叉树结构相同，两端点间会出现一些延伸到这一层的 null 节点，这些 null 节点也计入长度。

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
right(right) {}
 * };
 */
class Solution {
public:
    static const int MAXN = 3001;

    TreeNode* nq[MAXN];
    unsigned  long long iq[MAXN]; // 避免操作编号时溢出
    int l, r;

    int widthOfBinaryTree(TreeNode* root) {
        int ans = 1;
        int size, curWei;
        l = r = 0;
```

```
        nq[r] = root;
        iq[r++] = 1ULL; // 记录节点编号

        // 队列中还有节点
        while (l < r) {
            // 更新最大特殊宽度
            size = r - l;
            curWei = (int)(iq[r - 1] - iq[l] + 1);
            ans = curWei > ans ? curWei : ans;

            // 弹出栈内节点,并添加子节点
            for (int i = 0; i < size; i++) {
                TreeNode* cur = nq[l];
                unsigned long long id = iq[l++];
                if (cur->left != nullptr) {
                    nq[r] = cur->left;
                    iq[r++] = id * 2ULL; // 记录节点编号
                }
                if (cur->right != nullptr) {
                    nq[r] = cur->right;
                    iq[r++] = id * 2ULL + 1ULL; // 记录节点编号
                }
            }
        }

        return ans;
    }
};
```

4. 104. 二叉树的最大深度(*)

> 二叉树的最大深度是指从根节点到最远叶子节点的最长路径上的节点数。

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
right(right) {}
 * };
 */
class Solution {
public:
    int maxDepth(TreeNode* root) {
        return root == nullptr ?
        0 : max(maxDepth(root->left), maxDepth(root->right)) + 1;
    }
};
```

5. 111. 二叉树的最小深度(*)

> 最小深度是从根节点到最近叶子节点的最短路径上的节点数量。

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    int minDepth(TreeNode* root) {
        // 回溯条件
        if (root == nullptr) {
            return 0;
        }
        // 叶节点
        if (root->left == nullptr && root->right == nullptr) {
            return 1;
        }

        // 若无左节点或右节点
        // 应设子树深度为无穷
        // 避免误认该节点为叶节点
        int lDep = INT_MAX;
        int rDep = INT_MAX;
        //递归左右枝深度
        if (root->left != nullptr) {
            lDep = minDepth(root->left);
        }
        if (root->right != nullptr) {
            rDep = minDepth(root->right);
        }
        // 当前节点深度
        return min(lDep, rDep) + 1;
    }
};
```

6. 297. 二叉树的序列化与反序列化(***)

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
```

```
 *     TreeNode(int x) : val(x), left(NULL), right(NULL) {}
 * };
 */
class Codec {
public:

    // Encodes a tree to a single string.
    string serialize(TreeNode* root) {
        string ans;
        treef(root, ans);
        return ans;
    }
    void treef(TreeNode* root, string &s) {
        if (root == nullptr) {
            s+="#,";
        } else {
            s += to_string(root->val) + ",";
            treef(root->left, s);
            treef(root->right, s);
        }
    }

    // Decodes your encoded data to tree.
    // 按字符分割字符串
    vector<string> split(const string &s, char sign) {
        vector<string> res;
        string temp;
        for(char c : s) {
            if (c == sign) {
                if (!temp.empty()) {
                    res.push_back(temp);
                    temp.clear();
                }
            } else {
                temp += c;
            }
        }
        return res;
    }

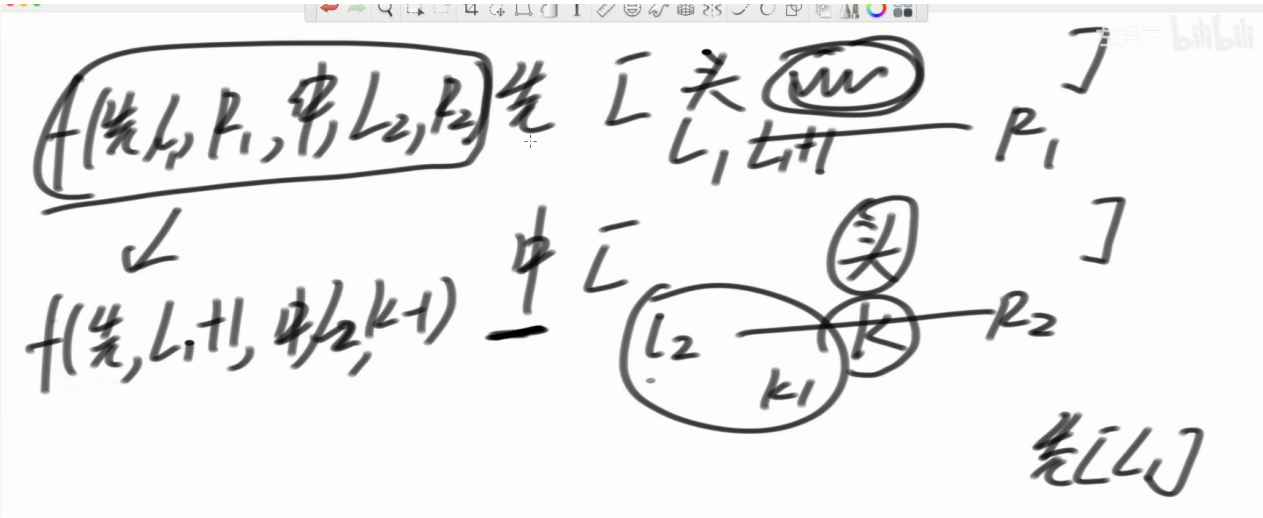
    int cnt;
    TreeNode* deserialize(string data) {
        vector<string> vals = split(data, ',');
        cnt = 0;
        return strToTree(vals);
    }

    // 递归反序列化
    TreeNode* strToTree(vector<string> &vals) {
        string val = vals[cnt++];
        if (val == "#") {
            return nullptr;
        } else {
```

```
            TreeNode* head = new TreeNode(stoi(val));
            head->left = strToTree(vals);
            head->right = strToTree(vals);
            return head;
        }
    }
};

// Your Codec object will be instantiated and called as such:
// Codec ser, deser;
// TreeNode* ans = deser.deserialize(ser.serialize(root));
```

7. 用先序和中序遍历结果建二叉树



105. 从前序与中序遍历序列构造二叉树(**)

> 给定两个整数数组 `preorder` 和 `inorder` ，其中 `preorder` 是二叉树的**先序遍历**， `inorder` 是同一棵树的**中序遍历**，请构造二叉树并返回其根节点。

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        if (preorder.empty() || inorder.empty() ||
        preorder.size() != inorder.size()) {
            return nullptr;
        }

```

```
        // 用哈希表记录中序遍历的索引
        unordered_map<int, int> dict;
        for (int i = 0; i < inorder.size(); i++) {
            dict.insert({inorder[i], i});
        }

        return genTree(preorder, 0, preorder.size()-1, inorder, 0,
inorder.size()-1, dict);
    }

    // 递归生成二叉树
    // l1, l2:先序遍历范围
    TreeNode* genTree(vector<int>& pre, int l1, int r1,
    vector<int>& in, int l2, int r2, unordered_map<int,int>&dict) {
        if (l1 > r1) { // 无效数组
            return nullptr;
        }
        TreeNode* head = new TreeNode(pre[l1]); // 头节点为先序遍历的第一个
        if (l1 == r1) { // 单个节点
            return head;
        }

        int k = dict[pre[l1]]; // 获取中序遍历的索引
        // 递归构造左右子树
        // pre : l1(........)[.......r1]
        // in  : (l2......)k[........r2]
        // (...)是左树对应，[...]是右树的对应
        head->left = genTree(pre, l1 + 1, l1 + k - l2, in, l2, k - 1, dict);
        head->right = genTree(pre, l1 + k - l2 + 1, r1, in, k + 1, r2, dict);
        return head;
    }
};
```

8. 958. 二叉树的完全性检验(**)

> 在一棵 **完全二叉树** 中，除了最后一层外，所有层都被完全填满，并且最后一层中的所有节点都尽可能靠左。

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
right(right) {}
 * };
 */
class Solution {
public:
    enum {MAXN = 101};
```

```
    bool isCompleteTree(TreeNode* root) {
        if (root == nullptr) {
            return false;
        }
        TreeNode* q[MAXN];
        int l, r;
        l = r = 0;
        q[r++] = root;
        bool leaf = false;
        // 队列不为空
        while (l < r) {
            TreeNode* cur = q[l++];
            // 有左无右 ， 或出现左右不满后不是叶节点的
            if ((cur->left == nullptr && cur->right != nullptr)
            || (leaf && (cur->left != nullptr || cur->right != nullptr))) {
                return false;
            }
            if (cur->left != nullptr) {
                q[r++] = cur->left;
            }
            if (cur->right != nullptr) {
                q[r++] = cur->right;
            }
            if (cur->left == nullptr || cur->right == nullptr) {
                leaf = true; // 子树不满，后序必须全为叶节点
            }
        }
        return true;
    }
};
```

9. 222. 完全二叉树的节点个数(*)

> 给你一棵 **完全二叉树** 的根节点 `root` ，求出该树的节点个数。

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
right(right) {}
 * };
 */
class Solution {
public:
    int countNodes(TreeNode* root) {
        if (root == nullptr) {
            return 0;
        }
```

```
        return recursion(root, 1, mostleft(root, 1));
    }

    // 递归求完全二叉树的节点
    int recursion(TreeNode* cur, int level, int h) {
        // base case
        if (level == h) {
            return 1;
        }

        if (mostleft(cur->right, level + 1) == h) {
            // 说明左树是是满节点的完全二叉树
            // 以当前节点为根的数的节点个数:
            // 1 + 左 + 右; 左=pow(2, height)
            return (1 << (h - level)) + recursion(cur->right, level + 1, h);
        } else {
            // 右树深度没到底，因为给的节点是完全二叉树右树是left_hight - 1 的满节点树
            return recursion(cur->left, level + 1, h) + (1 << (h - level - 1));
        }
    }

    //找最左节点深度
    int mostleft(TreeNode* head, int level) {
        while (head != nullptr) {
            level++;
            head = head->left;
        }
        return level - 1;
    }
};
```

10. [236. 二叉树的最近公共祖先](#)(**)

> 用递归找两个节点最近公共祖先

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode(int x) : val(x), left(NULL), right(NULL) {}
 * };
 */
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        // 找到一个p,q节点返回
        if (root == nullptr || root == p || root == q) {
            return root;
        }
        TreeNode* l = lowestCommonAncestor(root->left, p, q);
        TreeNode* r = lowestCommonAncestor(root->right, p, q);
```

```
19
20          if (l == nullptr && r == nullptr) {
21              // 都没搜到返回空
22              return nullptr;
23          } else if (l != nullptr && r != nullptr) {
24              // 都搜到返回当前节点
25              return root;
26          }
27          // 只搜到一个则返回搜到的节点
28          return l != nullptr ? l : r;
29      }
30  };
```

11. 235. 二叉搜索树的最近公共祖先(**)

> 利用二叉搜索树的特性

```
1   /**
2    * Definition for a binary tree node.
3    * struct TreeNode {
4    *     int val;
5    *     TreeNode *left;
6    *     TreeNode *right;
7    *     TreeNode(int x) : val(x), left(NULL), right(NULL) {}
8    * };
9    */
10
11  class Solution {
12  public:
13      TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
14          // root从上到下
15          // 如果先遇到了p，说明p是答案
16          // 如果先遇到了q，说明q是答案
17          // 如果root在p~q的值之间，不用管p和q谁大谁小，只要root在中间，那么此时的root就是答案
18          // 如果root在p~q的值的左侧，那么root往右移动
19          // 如果root在p~q的值的右侧，那么root往左移动
20          while (root != p && root != q) {
21              if (min(p->val,q->val) < root->val && root->val < max(p->val,q->val))
    {
22                  break; // root在p~q之间
23              }
24              // root值在p~q右侧,左移
25              // root值在p~q左侧,右移
26              root = root->val > max(p->val,q->val) ? root->left : root->right;
27          }
28          return root;
29      }
30  };
```

12. 113. 路径总和 II(**)

> 恢复现场

```cpp
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    vector<vector<int>> pathSum(TreeNode* root, int targetSum) {
        vector<vector<int>> ans;
        if (root != nullptr) {
            vector<int> path;
            recursion(root, ans, path, targetSum, 0);
        }
        return ans;
    }

    // 递归回溯
    void recursion(TreeNode*root, vector<vector<int>>&ans, vector<int>&path, int aim, int sum) {
        // 叶节点
        if (root->left==nullptr && root->right==nullptr) {
            if (aim == sum + root->val) {
                path.push_back(root->val);
                ans.push_back(path);
                path.pop_back(); // 恢复现场
            }
        } else {
        // 不是叶节点
            // 递归左右子树
            path.push_back(root->val);
            if (root->left != nullptr) {
                recursion(root->left, ans, path, aim, sum + root->val);
            }
            if (root->right != nullptr) {
                recursion(root->right, ans, path, aim, sum + root->val);
            }
            path.pop_back();
        }
    }
};
```

13. [110. 平衡二叉树](#)(*)

> 验证平衡二叉树

```cpp
/**
```

```
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    bool isBalanced(TreeNode* root) {
        bool balance = true;
        height(root, balance);
        return balance;
    }

    int height(TreeNode* root, bool&balance) {
        // base case
        if (!balance || root == nullptr) {
            return 0;
        }
        int lh = height(root->left, balance);
        int rh = height(root->right, balance);
        if (abs(lh - rh) > 1) {
            balance = false;
        }
        return max(lh,rh) + 1;
    }
};
```

14. [98. 验证二叉搜索树](#)(**)

> **有效** 二叉搜索树定义如下：
>
> - 节点的左子树只包含 **严格小于** 当前节点的数。
> - 节点的右子树只包含 **严格大于** 当前节点的数。
> - 所有左子树和右子树自身必须也是二叉搜索树。

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
```

```cpp
class Solution {
public:
    bool isValidBST(TreeNode* root) {
        if (root == nullptr) {
            return true;
        }
        // 中序遍历
        stack<TreeNode*> ast;
        TreeNode* pre = nullptr;
        // 栈不为空
        while (root != nullptr || !ast.empty()) {
            // 左支进栈
            if (root != nullptr) {
                ast.push(root);
                root = root->left;
            } else {
                root = ast.top();
                ast.pop();
                // 验证
                if (pre != nullptr && pre->val >= root->val) {
                    return false;
                }
                pre = root;
                root = root->right;
            }
        }
        return true;
    }
};
```

15. 669. 修剪二叉搜索树(**)

> 用递归修建二叉搜索树，使节点值符合范围

```cpp
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    TreeNode* trimBST(TreeNode* root, int low, int high) {
        // base case
        if (root == nullptr) {
            return nullptr;
        }
```

```
        // 比最小边界小，左树也不保留，递归右树
        if (root->val < low) {
            return trimBST(root->right, low, high);
        // 比最大边界大
        } else if (root->val > high) {
            return trimBST(root->left, low, high);
        }
        // 当前节点保留
        root->left = trimBST(root->left, low, high);
        root->right = trimBST(root->right, low, high);
        return root;
    }
};
```

16. 337. 打家劫舍 III(**)

> 维护全局变量实现`树形dp`;

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    // 全局变量,表示当前节点的收益
    int yes;
    int no;
    int rob(TreeNode* root) {
        dp(root);
        return max(yes, no);
    }


    void dp(TreeNode* cur) {
        // base case
        if (cur == nullptr) {
            yes = 0;
            no = 0;
        } else {
            int cy = cur->val;
            int cn = 0;
            // 递归左树
            dp(cur->left);
            cy += no;
            cn += max(yes, no);
```

```
            // 递归右树
            dp(cur->right);
            cy += no;
            cn += max(yes, no);
            // 维护全局变量
            yes = cy;
            no = cn;
        }
    }
};
```